

RET-LLM：为大型语言模型开发通用读写存储器

注：本概念文件概述了一种初始方法，现已在 MEMLLM 中得到发展和全面评估。

Ali Modarressi^{(1), (2)*} Ayyoob Imani^{(1), (2)*} Mohsen Fayyaz³ Hinrich Schütze^{(1), (2)}

⁽²⁾慕尼黑大学信息与语言处理中心，德国 ²慕尼黑机器学习中心，德国 (³微软，柏

林，德国

{amodaresi, ayyoob}@cis.lmu.de

摘要

大型语言模型（LLM）通过其广泛的参数和全面的数据利用，极大地推动了自然语言处理（NLP）领域的发展。然而，现有的 LLM 缺乏专用的内存单元，限制了它们为各种任务明确存储和检索知识的能力。在本文中，我们提出了 RET-LLM 这一新颖的框架，它为 LLM 配备了一个通用的写入-读取内存单元，使其能够根据任务执行的需要从文本中提取、存储和调用知识。受戴维森语义学理论的启发，我们以三元组的形式提取和保存知识。存储单元的设计具有可扩展性、可聚合性、可更新性和可解释性。通过定性评估，我们证明了我们提出的框架在问题解答任务中优于基本方法。此外，我们的框架在处理基于时间的问题解答任务时表现出强大的性能，展示了其有效管理时间相关信息的能力。

1 简介

近年来，大型语言模型（LLMs）极大地推动了自然语言处理（NLP）领域的发展（Bubeck 等人，2023 年；Chowdhery 等人，2022 年；Touvron 等人，2023 年）。凭借其庞大的参数数量和对大量数据的访问能力，LLM 在各种任务中都表现出了非凡的准确性。然而，目前最先进的 LLM 缺乏专用的记忆单元。取而代之的是，它们被训练成根据上下文预测单词，将知识隐含在参数中，这与理想的记忆功能不同。

理想的记忆单元应具备某些特征。首先，它应该允许读取和

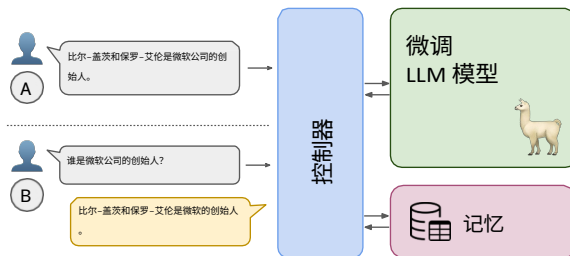


图 1：RET-LLM 概述。用户可能会提出 (A)：一个内容丰富的句子，我们的方法会将其中的有效信息存储在内存中；或者 (B)：一个问题，我们会利用之前保存的信息生成一个有效的答案。

写操作，使语言模型能够与存储的知识进行交互。可扩展性也至关重要，因为存储单元应适应知识不断发展的特性。此外，记忆单元不应仅限于文本文档；它应能够从数据库系统等不同来源获取知识。此外，还需要具备可解释性，使 LLM 能够深入了解解决特定任务所需的具体知识。最后，存储在存储单元中的信息应该是可聚合的，使模型能够结合多个文档中的相关信息。例如，LLM 应该能够列出多个文档中提到的一个国家的所有城市。

以往将记忆纳入 LLM 的尝试未能捕捉到完整的记忆特征。例如，（Zhong 等人，2022 年；Wu 等人，2022 年 2022 年）和（Cheng 等人，）将记忆退化为针对给定查询上下文重新获取相关文档的能力，并在生成答案时将其添加到上下文中。Park 等人（2023 年）只是在模拟环境中存储和再提取生成代理之前的观察和思考。

为了解决这些局限性，我们引入了 RET-LLM（Retentive LLM）解决方案。

* MemLLM (Modarressi et al., 2024)

* 同等贡献。

LLM 具有可扩展、可更新、可解释和可聚合的内存模块。我们的建议是为语言模型配备一个内存模块，使其能够从文本中提取知识并保存起来，供将来参考。当面临一项任务时，语言模型可以查询记忆模块，以获取更多信息来支持其响应。内存模块支持更新，并可纳入来自 SQL 和非 SQL 数据库及电子表格等非文本来源的信息。此外，它还能聚合分散在庞大文档或多个文档中与特定概念相关的各种信息。

图 1 显示了 RET-LLM 的结构。它由三个部分组成：LLM、控制器和存储单元。我们采用 Alpaca Taori 等人（2023 年）最近发布的指令调整语言模型 (LLM)，并设计了一个微调过程，使其具备以下能力：信息提取、信息查询和基于事实的答案生成。

信息提取需要从有信息的句子中识别和提取<概念1、关系、概念2>形式的三元组。信息查询任务包括在遇到需要进一步信息的任务时，查询存储单元以获取与给定概念及其相关关系有关的更多信息。最后，基于事实的答案生成包括根据检索到的信息生成最终答案。基于三元组的存储方法从戴维森语义学（Davidson, 1967）的理论框架中汲取灵感，该框架为使用类似于<事件、主语、宾语>的三元组结构来表示句子中描述的概念提供了基础。

存储模块存储三连音及其矢量表示。在检索过程中，它首先搜索与查询文本完全匹配的内容，如果没有找到完全匹配的内容，则根据向量表示进行模糊搜索。为了实现高效的模糊搜索和检索，我们采用了基于 LSH 的矢量哈希算法。控制器充当界面，自动处理用户、LLM 和内存模块之间的交互，确保与智能聊天系统的无缝交互体验。

与以前的方法相比，我们提出的方法具有多项优势。它使 LLM 能够明确地存储和检索知识，这一点至关重要

为现实世界的 NLP 应用服务。通过结合显式知识存储和检索，我们可以更好地了解这些模式的工作原理以及它们在解决任务时所依赖的知识。使用独立于 LLM 的外部存储单元确保了可扩展性和存储信息的易修改性。模糊搜索技术可以高效地检索相关信息，即使在没有完全匹配的情况下也是如此。以三元组形式存储信息有助于生成精确而全面的解决方案，尤其是在需要汇总数据时。最后，记忆模块可以方便地整合来自不同来源的信息，并记录随时间变化的事实。

通过使用问题交换实例进行定性评估，我们展示了 Alpaca-7B 等同类 LLM 无法重新生成正确答案的情况。我们发现，当模型能够获取生成有效答案所需的所有信息时，就会出现这种短板。然而，在我们提出的方法中，RET-LLM 在存储了从上下文中提取的知识后，无需重新输入上下文，就能显示出其回答问题的能力。我们还证明了 RET-LLM 可以处理基于时间的 QA 示例。因为 RET-LLM 配备了可修改的内存，可以处理时间事实。

2 相关作品

该领域的前人已探索通过检索相关文档并将其添加到任务上下文中，将相关上下文纳入大型语言模型（LLM）。Zhong 等人（2022 年）提出通过引入在训练过程中优化的可训练记忆单元，利用记忆增强训练 LLM。Wu 等人（2022 年）提出了记忆转换器（Memorizing Transformer），它可以在推理过程中关注较长的文档。这种方法将从转换层提取的（键、值）对存储在内存中，并在生成过程中检索相关对，将其添加到当前上下文中。（程等人）对每个文档进行编码、保存，并根据当前上下文检索相关文档。与这些方法相比，我们的方法具有更好的可扩展性，因为我们没有修改 LLM 的架构。相反，我们建议从文档中提取和保存信息，从而可以聚合从多个来源提取的信息。

这使我们能够提供更相关、更准确的检索信息，这些信息与正在解决的具体问题密切相关。

Park 等人（2023 年）在基因代理框架内使用了 LLM，以便于使用自然语言存储和动态检索代理经验的综合记录。不过，他们的架构与我们的架构存在本质区别。在 Park 的框架中，记忆组件是代理本身固有的一部分，而 LLM 则是一种外部工具，仅用于规划代理的行为。因此，LLM 无法控制在代理内存中存储和检索的具体内容。

Dhingra 等人（2022 年）为这一领域做出了贡献，他们设计了一个数据集，专门用于区分时间性和非时间性事实。他们建议在有时间标注的数据上训练语言模型，以增强其时间感知能力。这项工作与我们应对时态信息挑战的研究重点不谋而合。不过，在我们提出的解决方案中，我们通过引入可更新的记忆模块来应对这些挑战。

Schick 等人（2023 年）提出了一种方法，通过生成 API 调用来访问附加功能，如使用计算器执行任务，从而使 LLM 能够利用外部工具。在教授 LLM 使用外部工具方面，我们的工作与他们的方法有相似之处。不过，需要注意的是，我们的重点在于将更复杂、影响更大的工具（即内存模块）纳入其中，因为内存模块有可能对 LLM 的成果产生重大影响。

3 方法

我们的目标是设计一种 RET-LLM，用户可以在其中执行两个操作：（1）.提供一个或一系列信息语句，RET-LLM 应能记住其中的信息；（2）.提供一个或一系列信息语句，RET-LLM 应能记住其中的信息：提供一个或一系列信息性语句，RET-LLM 应能记住其中包含的信息。以前的方法是通过在所提供的文档上训练/微调 LLM，或者为文档创建一个向量表示并存储该表示来完成这项任务。（2）:提出相关问题，RET-LLM 将根据存储的记忆回答这些问题。所有这些操作都应在无缝环境中进行，用户只能使用自然语言进行交互。

我们的 RET-LLM 由以下三个主要部分组成

组成部分：（1）控制器（2）：微调 LLM &

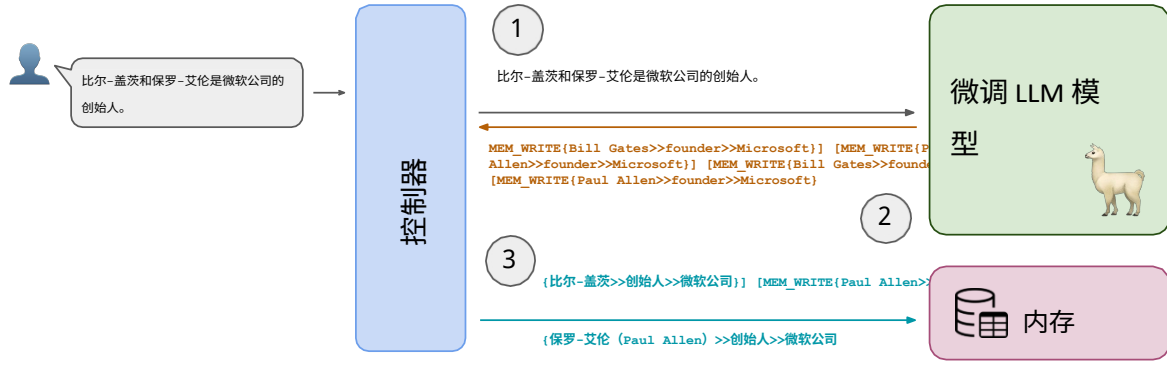
（3）:存储器。如图 1 所示，控制器控制着用户、LLM 和内存之间的信息流。LLM 充当处理单元，接收控制器发送的文本，并计算是否需要调用内存。受 Schick 等人（2023 年）的启发，由于 LLM 是通过文本进行操作的，我们通过实施基于文本的 API 模式，对内存调用进行了标准化。因此，LLM 可以生成内存 API 调用，控制器可以将 LLM API 调用应用到内存。在我们的设置中，内存使用三列表以三元组方式存储数据。这是以戴维森语义学（Davidson戴维森, 1967）的理论框架为基础的，在语义学中，句子中描述的概念可以存储在<第一参数、关系、第二参数>的结构中。

下面我们将详细介绍 RET-LLM。内存应用程序接口（memory-API），我们如何对 LLM 进行微调，使其能够执行这些调用，以及内存结构。

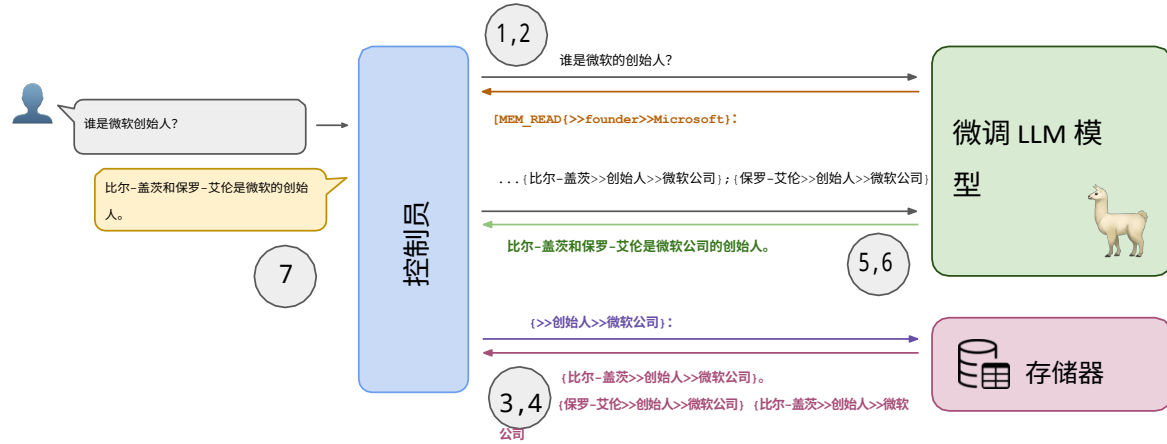
3.1 内存结构

每个三元组定义了两个参数之间的关系，格式如下： $(t_1, t_2, t_{(3)})$ 其中 t_1 是第一个参数， t_2 是关系， $t_{(3)}$ 是关系中的第二个参数。例如，在句子中"马克-扎克伯格是 Meta 公司的首席执行官"，可以提取的信息三元组是：（马克-扎克伯格、首席执行官、Meta 公司）。

为了存储这些三连音，我们使用了一个三列表，其中每一列都与三连音的每个部分相关联。在保存文本的同时，我们还存储了平均表示法，以便内存模块也能处理具有语义相似单词的查询。如果存储模块无法在表中找到准确的文本，它就会通过比较查询文本的向量表示和数据集中已存储的文本片段的向量表示，来检查是否有相似的文本。因此，对于每个 t_i ，LLM 所检索到的平均表示法 $(h_{(AV)}(t_i))$ 都会存储在位置敏感散列（LSH）表中。利用 LSH 的原因是为了减少查找相似表示所需的计算量。如果没有给定查询表示的哈希表，就必须计算所有存储表示的距离，这将是一项计算量巨大的任务。



(a) 内存写入方案：(1) 控制器将输入传递给 LLM (2)，LLM 生成适当的内存写入调用。(3) 控制器将数据（及其平均值）提供给要存储的内存。



(b) 内存读取方案：(1) 控制器将问题传递给 LLM (2)，LLM 生成适当的内存读取调用。(3) 控制器根据 LLM 给出的搜索条件对内存进行查询。(4) 内存返回查询结果，查询结果 (5) 返回 LLM。(6) LLM 利用查询结果生成问题的答案，(7) 将答案返回给用户。

图 2：基于读取和写入的输入过程可视化。

处理内存查询在内存查询中，应提供一个或两个三元组参数作为输入：

$$Q \in \{ \langle q_1 \rangle, \langle q_2 \rangle, \langle q_3 \rangle, \langle q_{(1)}, q_2 \rangle, \langle q_1, q_3 \rangle, \langle q_{(2)}, q_3 \rangle \}$$

其中， q_i 是存储元组中第 i 个参数的搜索词。在检索查询结果之前，要对每个搜索词进行检查 对于给定的 Q ，内存首先会检查搜索词 (q_i) 在存储表中是否完全匹配。如果 q_i 不存在于存储项中，我们会使用其平均表示法 $\mathbf{h}_{(AV)}(G)(q_i)$ 和 LSH 表来查找在以下条件中完全匹配的替代项 ($\tilde{q}_i$)

出内存表。LSH 表可能没有为给定的表示法找到一个替代词，因此查询不会有结果： $Q \rightarrow \emptyset$ 。在任何情况下（完全匹配或相似匹配），查询在数据表中都可能有多多个匹配结果 ($q_i = t(i)$)。在这种情况下，所有结果三元组都将作为查询输出

3.2 内存-API 和数据流

为了实现内存与 LLM 之间的通信，我们为内存读写函数设计了一个 API 架构。通过这个应用程序接口，控制员可以了解 LLM 调用内存的时间以及应该传递哪些参数。根据上一节讨论的三元组，两种内存调用方式如下：

- $[MEM_WRITE\{t_{(1)} t_{(2)} t_{(3)}\}]$ ：该结构用于存储三元组 $\langle t_1, t_2, t_3 \rangle$ 。根据提示，LLM 可以依次生成多个写调用，以存储从文本中提取的多个三连串。
- $[mem_read\{ _ _ \}]$ ： $\{t_{(1)} t_{(2)} t_{(3)}; \dots\}$:在返回。

如 API 所示，在内存读取中，有三个占位符，根据 Q 值，至少有一个占位符应填入搜索条件。根据内存中的查询结果，可以返回一个或一系列三元组，如高亮显示部分所示。

图 2 演示了 RET-LLM 如何使用内存应用程序接口进行操作。根据用户的输入，RET-LLM 要么从内存读取信息，要么向内存写入信息。如果用户提示一条信息性语句（或者最好是一份完整的文档），那么就会出现内存写入的情况。另一方面，如果输入的是一个问句，我们就认为这是内存读取的情况。在这两种情况下，用户输入都是传递给 RET-LLM 的第一个输入。

根据给定的输入，LLM 推断并生成相关的 API 调用。在内存写入的情况下，API 调用生成后，控制器会检测到它，并使用给定的参数调用内存存储函数。内存接收三元组格式的数据，并将其存储起来，以备将来使用。如果 LLM 生成了内存读取调用，控制器也会检测到该调用，并暂停模型序列生成以进行内存检索。它使用读取调用中给出的参数作为查询条件，并将其传递给存储器。内存会列出所有以给定的搜索条件（或根据第 3.1 节的规定，以语义相似的条件）为特征的已存储三元组，并将结果返回给控制器。利用本节开头讨论的应用程序接口，读取结果会在调用后列出，以便 LLM 利用它们生成自然发音的答案。答案生成后，将返回给用户。

由于控制器处于用户和 LLM 之间，因此可以隐藏整个内存-API 架构。这样，用户就能感受到端到端的简单语言建模体验，而无需了解背后的内存功能。

3.3 微调 LLM

在这一部分，我们将讨论如何对 LLM 进行微调，使其能够生成内存应用程序接口调用。最终，LLM 应该能够根据输入内容检测出应该调用哪种类型的内存调用（读取或写入）。如第 3.2 节所述，根据内存功能的不同，LLM 的输入可能具有前面讨论过的两种结构之一。因此，LLM 应该能够生成并处理这种 API，以存储或读取相关信息。为此，我们开发了一个合成数据集来训练 LLM。合成任务是学习所讨论的人与相应公司之间的关系。根据存储的信息，RET-LLM 应该能够

能够回答与人物、公司或关系相关的任何问题。

我们使用一组名字和姓氏生成一个合成人口，称为 P 。该人口 $per \in P$ 中的每个人只能与以下列表中的一个组织有关系： $rel \in R = \{\text{employment, manager, investor, founder, customer}\}$ with an organization $org \in O$ 其中， O 是一组公司名称。因此，每个三元组都是 (per, rel, org) 。例如： $\langle \text{Dominick Alphonso, employment, BMW} \rangle$ 。¹根据这个三元组，我们可以建立三个特定于三元组的问题：

- $Q = (per)$ ，例如 "谁是多米尼克-阿方索？"
- $Q = (per, org)$ ，例如 "多米尼克-阿方索与宝马有什么关系？"
- $Q = (per, rel)$ ，例如 "多米尼克-阿方索受雇于哪家公司？"

而上述所有问题的答案都应该是 "多米尼克-阿方索受雇于宝马公司"。除了这些问题之外，还可以提出与多个三胞胎相关的其他三种类型的问题：

- $Q = (rel)$ ，例如 "员工是谁？"
- $Q = (org)$ ，如 "谁与宝马公司有关？"
- $Q = (rel, org)$ ，例如 "谁受雇于宝马公司？"

与前三个问题不同的是，每个问题的答案都可能与多人有关。对于每个问题，我们都希望模型能在不提供任何额外信息的情况下（例如，在未被问及的情况下说明就业公司）回答问题。我们使用表 1 中列出的模板，根据记忆 API 从这些问题中创建训练数据实例。在微调过程中，问题、API 查询（使用 MEM_READ 命令）、API 响应和答案被整合为 LLM 的数据输入。不过，语言建模损失仅适用于 API 查询和回答部分。因为这两个部分是 LLM 根据控制器提供的其他两个部分（问题和 API 响应）生成的文本序列。

¹ 虽然公司名称是真实的，但人名完全是随机生成的。无意或不应推断出与实际人物的识别。

查询类型	问题	API 查询	API 响应	应答
$\langle per \rangle$	谁是 per?	$\{per'''\}$:	$\{per \text{ "rel "org} \}$:	per 是 org 的关系
$\langle per, org \rangle$	per 与 org 有什么关系? 谁与	$\{per'''\text{org} \}$:	$\{per \text{ "rel "org} \}$:	。
$\langle per, rel \rangle$	org 有关?	$\{per \text{ "rel" } \}$:	$\{per \text{ "rel "org} \}$: $\{per \text{ "rel "org} \}$	$[per_1, per_2, \dots]$ is/are related to org $[per_1,$
$\langle org \rangle$	谁是 rel? 谁是	$\{'''\text{org} \}$:	$\{per_{(1)}\text{-rel}_{(1)}\text{org} \}; \{per_{(2)}\text{-rel}_{(2)}\text{org} \}; \dots$	$per_2, \dots]$ is/are rel.
$\langle rel \rangle$	org 的关系人?	$\{'''\text{rel} \}$:	$\{per_{(1)}\text{-rel "org}_{(1)} \}; \{per_{(2)}\text{-rel "org}_{(2)} \}; \dots$	$[per_1, per_2, \dots]$ is/are rel to org.
$\langle org, rel \rangle$		$\{'''\text{rel "org} \}$:	$\{per_{(1)}\text{-rel "org}_{(1)} \}; \{per_{(2)}\text{-rel "org}_{(2)} \}; \dots$	

表 1：用于微调的内存读取数据示例。前三类问题基于单个三元组，因此 API 响应将只有一个三元组。然而，如 API-Resonse 所示，后三类问题的内存中可能存储有多个相关的小提示。因此，答案应将三连音数据合并成一句话。 $[per_1, per_2, \dots]$ 是以自然方式按顺序写出的名称占位符。例如"Dirk Alosa、Ty Baumkirchner 和 Vera Bayless

三胞胎	声明	API 写调用
$[\langle per_1, rel, org \rangle, \langle per_2, rel, org \rangle, \dots]_o$	$[per_1, per_2, \dots]$ is/are rel to org.	$[MEM_WRITE\{per_{(1)}\text{-rel}_{(1)}\text{org} \}][MEM_WRITE\{per_{(2)}\text{-rel}_{(2)}\text{org} \}]\dots$

表 2：用于微调的内存写入数据示例结构。

由于我们也需要有 MEM_WRITE 调用的信息示例，因此我们使用了类似的策略，即使用之前定义的人口、组织和关系 (P 、 Q 、 R)。根据内存应用程序接口，在内存写入场景中，RET-LLM 接收到一个句子，其中包含关系信息，然后 LLM 应生成相应的内存写入调用。在我们的数据集中，我们选择建立一个例子来说明与同一家公司有相同关系的多人： $(per_i, rel, org)_o$ 。表 2 中显示了备忘录写入数据示例的模板。与基于问题的示例类似，语句和 API 调用被连接起来形成完整的输入序列。此外，损失函数仅适用于 API 部分，因为第一部分由控制器提供。

我们选择使用指令跟随 Alpaca-7B 模型 (Taori 等人, 2023 年) 作为微调的基础模型。为了在资源有限的情况下进行训练，我们使用了低阶适应 (Low-rank adap-tation, LoRA) (Hu 等人, 2022 年)。²这种参数高效的措施使我们能够在单个 A6000 48GB GPU 上对基础模型进行微调。

4 定性结果

在本部分中，我们将介绍多个评估示例的内部流程和最终输出结果。这些示例是按照第 3.3 节所述的相同程序生成的。首先，为了证明我们的方法的重要性，我们提供了相同的示例来说明我们的方法。

我们的基础模型 (Alpaca-7B) 在零镜头设置中的表现。输入将是任务的简短指令、例句中的信息句，最后是问题。如图 3 所示，经过指令调整模型得出的零拍摄结果显然是不正确的。虽然该模型确实掌握了上下文中的所有信息，但仍然产生了错误的反应。

在同一个例子中，RET-LLM 首先将从示例中提取的三元组存储到内存中。存储提取的关系后，RET-LLM 即使在没有输入信息的情况下也能对同一问题做出回答。在内存应用程序接口和内存本身的帮助下，可以找到相关的三元组。在将查询结果附加到内存调用后，LLM 就能做出正确的回答。

我们的方法的一个潜在用例是回答具有时间背景的问题。例如，美国总统每 4 到 8 年更换一次。普通的 PLM 模型会根据自己的训练数据回答有关总统职位的问题。虽然模型的重新训练或参数编辑有其自身的困难，但我们的方法可以为这一问题提供一个简单且可解释的解决方案 (图 4)。

5 结论与未来工作

在这项工作中，我们介绍了一种能够存储信息并在进一步使用时检索信息的 RET-LLM 方法。通过基于三元组的内存结构，信息被存储在具有已知关系的两个参数之间的关系中。内存可通过内存应用程序接口 (memory-API) 使用，该接口可生成

² 使用 LoRA 对基于骆马的模型进行微调的代码可在以下网址获得：github.com/tloen/alpaca-lora

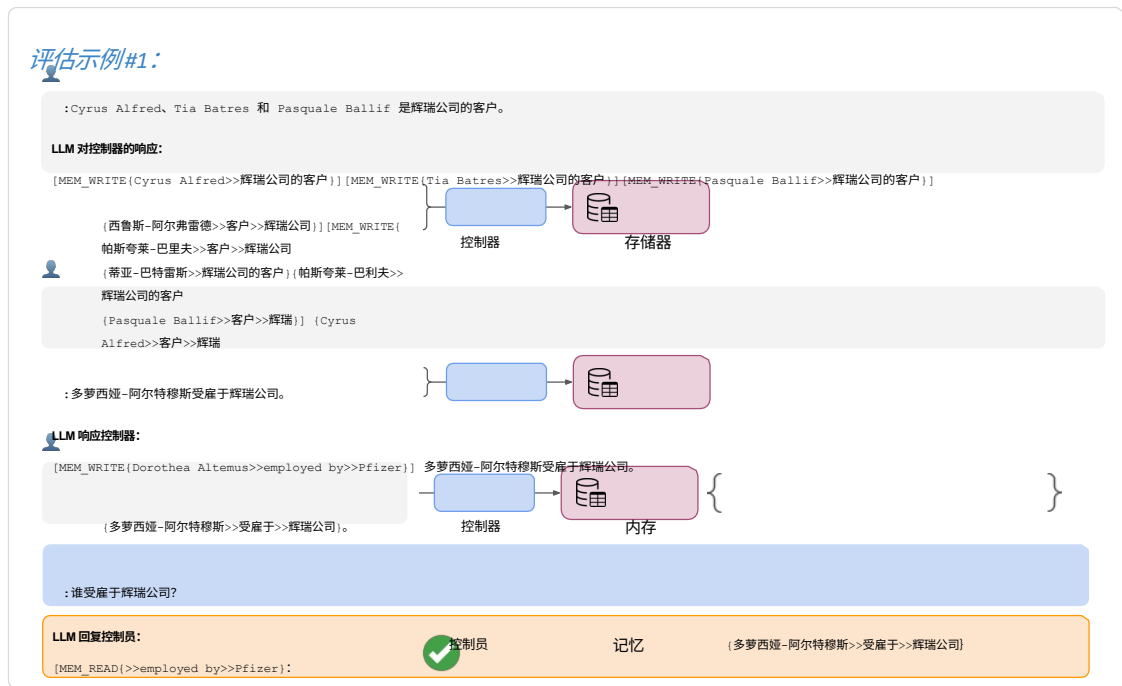


图 3：在零镜头设置中结果不正确，而在我们的方法中结果正确的示例。请注意，在 "零镜头 "设置中，模型可以直接访问其输入中回答问题所需的信息，但最终得到的答案仍然是错误的。不过，在我们的方法中，用户的每条提示都可以单独提供给 RET-LLM。附录图 5 中提到了另一个例子。



图 4：提出一个需要时间上下文的问题通常会得到一个过时的答案，如图中的 Alpaca 所示。然而，在我们的 RET-

LLM 中，借助可修改的内存，只须提供更新的内存条目，就能回答这些问题。

通过微调的 LLM。通过控制器，所有组件可以相互通信，而用户可以在不知道背后过程的情况下与控制器进行交互。我们已经证明，在一些问题解答示例中，LLM 在没有输入上下文信息的情况下也能生成正确的 API 调用。由于这项工作仍在开发中，我们将在下一次修订中加入更详细的实证评估，尤其是在真实数据集上。我们还将努力改进我们的微调方法，使其更加通用，从而能够处理更多类型的信息关系。

参考文献

Sébastien Bubeck, Varun Chandrasekaran, Ronen Eldan, Johannes Gehrmke, Eric Horvitz, Ece Kamar, Peter Lee, Yin Tat Lee, Yuanzhi Li, Scott Lundberg, et al. 人工通用智能的火花：早期 gpt-4 实验. *arXiv 预印本 arXiv:2303.12712*.

Xin Cheng, Yankai Lin, Dongyan Zhao, and Rui Yan. 具有插件式知识记忆的语言模型.

Aakanksha Chowdhery, Sharan Narang, Jacob Devlin, Maarten Bosma, Gaurav Mishra, Adam Roberts, Paul Barham, Hyung Won Chung, Charles Sutton, Sebastian Gehrmann, Parker Schuh, Kensen Shi, Sasha Tsvyashchenko, Joshua Maynez, Abhishek Rao, Parker Barnes, Yi Tay, Noam Shazeer, Vinodkumar Prabhakaran, Emily Reif, Nan Du, Ben Hutchinson, Reiner Pope, James Bradbury, Jacob Austin, Michael Isard, Guy Gur-Ari, Pengcheng Yin, Toju Duke, Anselm Levskaya, Sanjay Ghemawat, Sunipa Dev, Henryk Michalewski, Xavier Garcia, Vedant Misra, Kevin Robinson, Liam Fedus, Denny Zhou, Daphne Ippolito, David Luan, Hyeontaek Lim, Barret Zoph, Alexander Spiridonov, Ryan Sepassi, David Dohan, Shivani Agrawal, Mark Omernick, Andrew M. Dai, Thanumalayan Sankaranarayanan, Pi-Lai, Marie Pellat, Aitor Lewkowycz, Erica Moreira, Rewon Child, Oleksandr Polozov, Katherine Lee, Zongwei Zhou, Xuezhi Wang, Brennan Saeta, Mark Diaz, Orhan Firat, Michele Catasta, Jason Wei, Kathy Meier-Hellstern, Douglas Eck, Jeff Dean, Slav Petrov, and Noah Fiedel. 2022. *Palm: 扩展语言修改用路径*.

唐纳德·戴维森. 1967. 行动句的逻辑形式, 重印于戴维森 (1980 年) 《行动与事件论文集》。

Bhuvan Dhingra, Jeremy R. Cole, Julian Martin Eisenschlos, Daniel Gillick, Jacob Eisenstein, and William W. Cohen. 2022. 作为时代知识库的模态时间感知语言. *计算语言学协会论文集*, 10: 257-273.

Edward J Hu, yelong shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. 2022. *LoRA: 大型的低阶适应语言模型*. *学习表征国际会议*.

Ali Modarressi, Abdullatif Köksal, Ayyoob Imani, Mohsen Fayyaz, and Hinrich Schütze. 2024. Mem-llm: *ArXiv preprint arXiv:2404.11672*.

Joon Sung Park, Joseph C O'Brien, Carrie J Cai, Meredith Ringel Morris, Percy Liang, and Michael S Bernstein. 2023. 生成代理: *arXiv preprint arXiv:2304.03442*.

Timo Schick, Jane Dwivedi-Yu, Roberto Dessi, Roberta Raileanu, Maria Lomeli, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. 2023. *Toolformer: 语言模型可以自学使用工具*. *arXiv 预印本 arXiv:2302.04761*.

Rohan Taori, Ishaan Gulrajani, Tianyi Zhang, Yann Dubois, Xuechen Li, Carlos Guestrin, Percy Liang, and Tatsunori B. Hashimoto. 2023. 斯坦福羊驼: 指令跟随的骆驼模型. https://github.com/tatsu-lab/stanford_alpaca

Hugo Touvron, Thibaut Lavril, Gautier Izacard, Xavier Martinet, Marie-Anne Lachaux, Timothée Lacroix, Baptiste Rozière, Naman Goyal, Eric Hambro, Faisal Azhar, Aurelien Rodriguez, Armand Joulin, Edouard Grave, and Guillaume Lample. 2023. *Llama: 开放而高效的基础语言模型*

吴雨怀, 马库斯-N-拉贝, 德莱斯利-哈钦斯和克里斯蒂安-塞格迪. 2022. 记忆变换器. *arXiv 预印本 arXiv:2203.08913*.

Zexuan Zhong, Tao Lei, and Danqi Chen. 2022. 利用记忆增强语言模型训练. *自然语言处理经验方法 2022 年会议论文集*, 第 5657-5673 页, 阿联酋阿布扎比. Association for Computational Linguistics.

A 额外评估示例

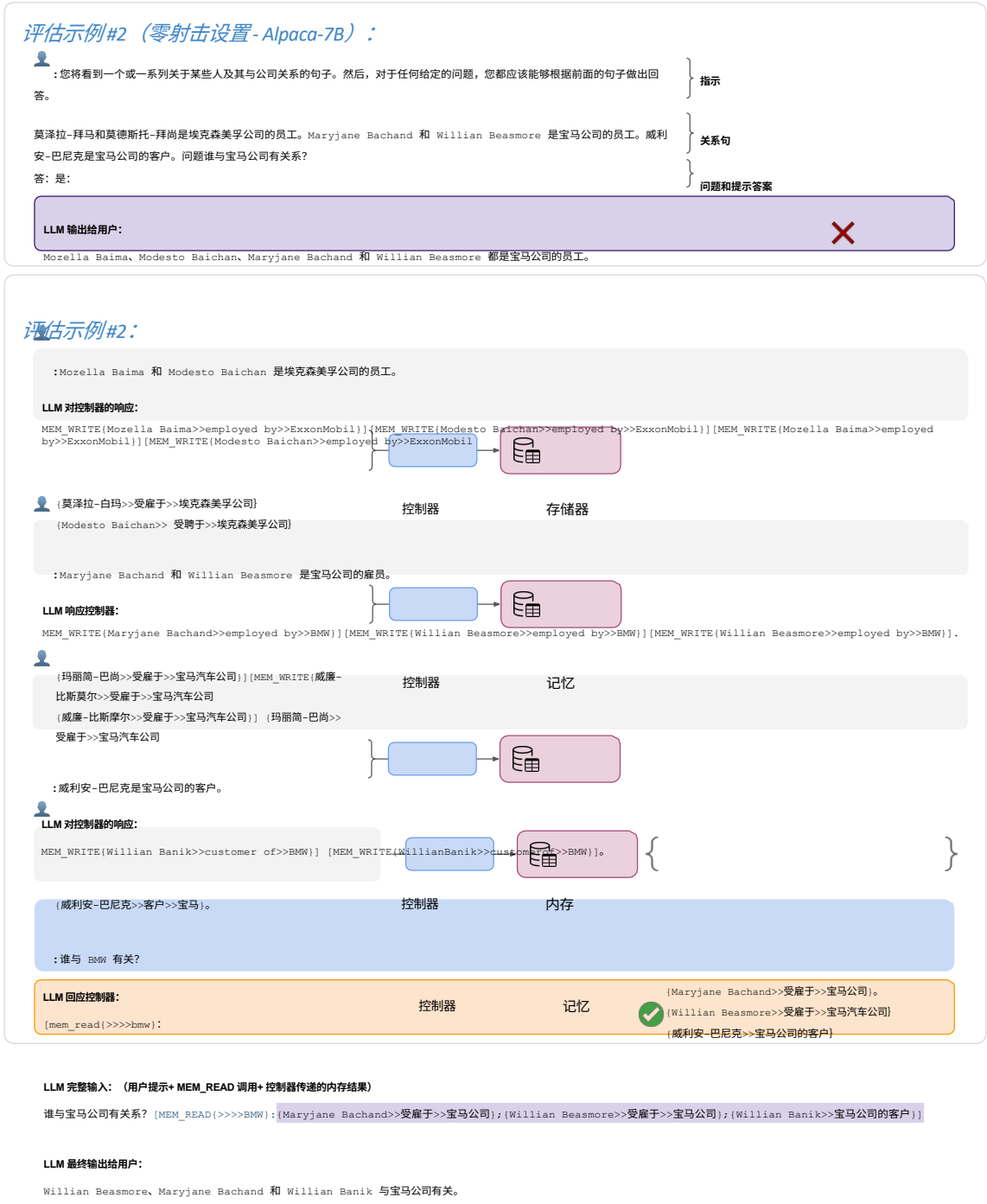


图 5：另一个评估示例，在零镜头设置中结果不正确，而在我们的方法中结果正确。